

```
print("Dhwaj Jain S012 AI&DS DL Expt 6")
```

```
Dhwaj Jain S012 AI&DS DL Expt 6
```

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.datasets import mnist
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
```

```
(x_train_mnist, _), (x_test_mnist, _) = mnist.load_data()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 — 0s 0us/step
```

```
x_train_mnist = x_train_mnist.reshape((len(x_train_mnist),
np.prod(x_train_mnist.shape[1:])))
x_test_mnist = x_test_mnist.reshape((len(x_test_mnist),
np.prod(x_test_mnist.shape[1:])))
x_train_mnist = x_train_mnist.astype('float32') / 255.0
x_test_mnist = x_test_mnist.astype('float32') / 255.0
print(f"Mnist Training data shape: {x_train_mnist.shape}")
print(f"Mnist Test data shape: {x_test_mnist.shape}")
```

```
Mnist Training data shape: (60000, 784)
Mnist Test data shape: (10000, 784)
```

```
pca_mnist = PCA(n_components=32)
x_train_pca_reduced = pca_mnist.fit_transform(x_train_mnist)
x_test_pca_reduced = pca_mnist.transform(x_test_mnist)
x_test_pca_reconstructed = pca_mnist.inverse_transform(x_test_pca_reduced)
mse_pca_mnist = mean_squared_error(x_test_mnist,
x_test_pca_reconstructed)
print(f"Reduced data shape (PCA on MNIST): {x_test_pca_reduced.shape}")
print(f"Reconstructed data shape (PCA on MNIST): {x_test_pca_reconstructed.shape}")
print(f"Mean Squared Error after PCA reduction on MNIST to 32 dimensions: {mse_pca_mnist}")
```

```
Reduced data shape (PCA on MNIST): (10000, 32)
Reconstructed data shape (PCA on MNIST): (10000, 784)
Mean Squared Error after PCA reduction on MNIST to 32 dimensions: 0.016828222200274467
```

```
input_dim_mnist = x_train_mnist.shape[1]
input_layer_mnist = Input(shape=(input_dim_mnist,))
encoded_mnist = Dense(128, activation='sigmoid')(input_layer_mnist) # Corrected: Pass input_layer_mnist here
latent_mnist = Dense(32, activation='sigmoid')(encoded_mnist) # Corrected: Pass output of encoded_mnist
decoded_mnist = Dense(128, activation='sigmoid')(latent_mnist) # Corrected: Pass output of latent_mnist
output_layer_mnist = Dense(input_dim_mnist, activation='sigmoid')(decoded_mnist) # Corrected: Pass output of decoded_mnist
autoencoder_mnist = Model(inputs=input_layer_mnist,
outputs=output_layer_mnist)
autoencoder_mnist.compile(optimizer='adam', loss='binary_crossentropy')
```

```
print("\nAutoencoder summary for MNIST:")
autoencoder_mnist.summary()
```

Autoencoder summary for MNIST:
Model: "functional_1"

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 784)	0
dense_6 (Dense)	(None, 128)	100,480
dense_7 (Dense)	(None, 32)	4,128
dense_8 (Dense)	(None, 128)	4,224
dense_9 (Dense)	(None, 784)	101,136

Total params: 209,968 (820.19 KB)
Trainable params: 209,968 (820.19 KB)

```
history_mnist = autoencoder_mnist.fit(
    x_train_mnist, x_train_mnist,
    epochs=20,
    batch_size=256,
    validation_data=(x_test_mnist, x_test_mnist),
    verbose=0
)
print("Autoencoder training for MNIST complete.")
x_test_autoencoder_reconstructed = autoencoder_mnist.predict(x_test_mnist)
mse_autoencoder_mnist = mean_squared_error(x_test_mnist,x_test_autoencoder_reconstructed)
print(f"Reconstructed data shape (Autoencoder on MNIST):{x_test_autoencoder_reconstructed.shape}")
print(f"Mean Squared Error after Autoencoder reduction on MNIST to 32 dimensions: {mse_autoencoder_mnist}")
```

Autoencoder training for MNIST complete.
313/313 1s 2ms/step
Reconstructed data shape (Autoencoder on MNIST):(10000, 784)
Mean Squared Error after Autoencoder reduction on MNIST to 32 dimensions: 0.020096253603696823

Start coding or [generate](#) with AI.